Smart Task Scheduler

with Dynamic Strategy Execution

Advanced Programming (OOP) Lab Project Report

Course	: Advanced Programming (OOP) Lab
Project Title	: Smart Task Scheduler with Dynamic Strategy Execution
Student Name	: Megha Das
Roll Number	: 25
Instructor Name	: Dr. Soumadip Biswas, Dr. Swagatam Basu and Dr.Susovan Jana
Semester	: Spring 2026

Repository: https://github.com/IEM-Gurukul/term-ii-project-submission-stellina18/tree/main

Contents

1 Problem Statement and Objectives **3**

1.1 Objectives 3

1.2 Target Users 3

2 System Architecture **3**

2.1 UML Class Diagram 4

2.2 Package Structure 5

3 OOP Concepts Applied **5**

3.1 Abstraction 5

3.2 Inheritance 5

3.3 Polymorphism 6

3.4 Exception Handling 6

3.5 Collections 7

4 Design Patterns **7**

4.1 Strategy Pattern 7

4.2 Observer Pattern 8

5 Code Snippets with Explanations **8**

5.1 Snippet 1: Abstract Task — Constructor Validation 8

5.2 Snippet 2: Polymorphic Urgency Score 9

5.3 Snippet 3: Strategy Pattern in SchedulerManager 9

5.4 Snippet 4: Observer Notification 10

5.5 Snippet 5: Custom Checked Exception 11

6 Data Persistence **12**

6.1 File Format: JSON 12

6.2 Save and Load Cycle 12

6.3 JSON Record Structure 12

7 Git Commit History **13**

8 Challenges and Learnings **13**

8.1 Challenges Encountered 13

8.2 Key Learnings 14

9 Conclusion **14**

Appendix B: Approval Email **15**

1. Problem Statement and Objectives

Students and professionals frequently face the challenge of managing multiple overlapping deadlines that vary in urgency, priority, and complexity. Most existing task management applications rely on manual sorting and static priority assignments. As a result, users often miss critical deadlines or experience unproductive workload spikes when conditions change dynamically.

The Smart Task Scheduler addresses this problem by modelling tasks through a structured object-oriented hierarchy. Each task encapsulates its own attributes — priority, deadline, estimated duration — and a pluggable scheduling mechanism automatically reorders tasks as conditions evolve, requiring no manual intervention from the user.

1.1. Objectives

- Model diverse task types (one-time, recurring, urgent) through a clean OOP hierarchy.
- Implement three interchangeable scheduling strategies switchable at runtime.
- Detect and report scheduling conflicts automatically using the Observer pattern.
- Persist task data across sessions using a custom JSON file handler (no external libraries).
- Demonstrate robust exception handling for invalid inputs and constraint violations.

1.2. Target Users

College students managing academic deadlines and early-career professionals tracking project commitments who need dynamic, automatic task reordering rather than manual lists.

2. System Architecture

The system follows a **layered object-oriented architecture** with four clearly separated concerns. This separation ensures that changes in one layer do not propagate unpredictably into another.

Layer	Responsibility
Model Layer	Defines the `Task` abstract class and three subclasses. Encapsulates all task-specific data and behaviour.
Strategy Layer	Provides three interchangeable sorting algorithms behind a common `SchedulingStrategy` interface.
Observer Layer	Contains `DeadlineConflictNotifier` and `WorkloadMonitor` that react to task list changes automatically.
Persistence Layer	`TaskFileHandler` reads and writes task data to `data/tasks.json` using only the standard Java library.

Table 1: Layered Architecture Summary

2.1. UML Class Diagram

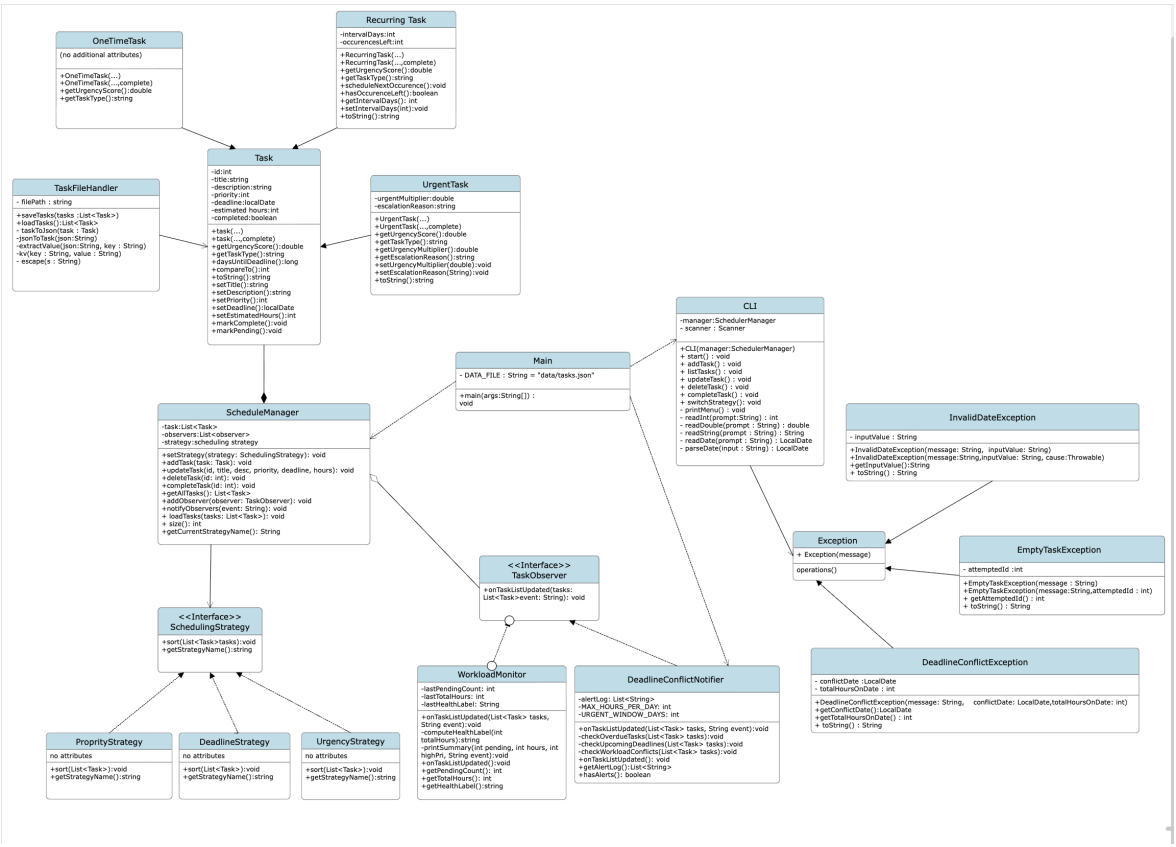

Figure 1: UML Class Diagram — Smart Task Scheduler

The diagram shows how `SchedulerManager` acts as the central coordinator: it holds a reference to the active `SchedulingStrategy`, maintains a list of `TaskObserver`

instances, and manages the master `Task` list. The three `Task` subclasses each inherit common behaviour while overriding polymorphic methods independently.

2.2. Package Structure

```
src/com/taskscheduler/
  model/        Task (abstract), OneTimeTask, RecurringTask, UrgentTask
  strategy/     SchedulingStrategy (interface), DeadlineStrategy,
                PriorityStrategy, UrgencyStrategy
  observer/     TaskObserver (interface), DeadlineConflictNotifier,
                WorkloadMonitor
  manager/      SchedulerManager
  persistence/  TaskFileHandler
  exception/    InvalidDateException, DeadlineConflictException,
                EmptyTaskException
  ui/           CLI
  Main.java
```

3. OOP Concepts Applied

3.1. Abstraction

Concept: Abstraction

Abstraction hides implementation complexity behind a well-defined contract. The abstract class `Task` defines *what* a task is without specifying *how* urgency is computed for each task type.

The `Task` class is declared `abstract` and mandates two abstract methods: `getUrgencyScore()` and `getTaskType()`. All shared behaviour — deadline validation, natural ordering via `Comparable`, CRUD accessors — is fully implemented in the base class and inherited by subclasses without duplication.

The `SchedulingStrategy` and `TaskObserver` interfaces provide additional layers of abstraction, allowing `SchedulerManager` to work with any sorting algorithm or notification listener without knowing its concrete type.

3.2. Inheritance

Three specialised subclasses extend `Task`:

- **`OneTimeTask`** — A simple non-recurring task. Urgency is driven entirely by deadline proximity and priority level.

- **`RecurringTask`** — Adds `intervalDays` and `occurrencesLeft` fields plus a `scheduleNextOccurrence()` method that advances the deadline automatically on completion.
- **`UrgentTask`** — Adds an `urgencyMultiplier` and an `escalationReason` string that amplify the task's scheduling weight above all other task types.

All three subclasses inherit `Task`'s constructor validation, `compareTo()`, and `toString()` logic, demonstrating code reuse without duplication.

3.3. Polymorphism

Concept: Polymorphism

`SchedulerManager` sorts a mixed list of `OneTimeTask`, `RecurringTask`, and `UrgentTask` objects using a single `getUrgencyScore()` call — each type responds differently at runtime.

The `UrgencyStrategy` class calls `task.getUrgencyScore()` uniformly. Because each subclass overrides this method, Java's dynamic dispatch invokes the correct implementation automatically:

- `OneTimeTask`: score $= p \times \left(1 + \frac{10}{d+1}\right)$
- `RecurringTask`: score $= p \times \left(1 + \frac{10}{d+1}\right) \times \frac{30}{\text{interval}}$
- `UrgentTask`: score $= p \times \left(1 + \frac{10}{d+1}\right) \times \text{multiplier}$

where p is priority and d is days until deadline.

3.4. Exception Handling

Three custom exception classes are implemented to model distinct failure modes:

Exception Class	When Thrown
`InvalidDateException`	User enters a past or unparseable date.
`DeadlineConflictException`	Adding or updating a task would exceed the 12 h/day workload cap.
`EmptyTaskException`	An operation targets a task ID that does not exist.

Table 2: Custom Exception Classes

All exceptions carry contextual fields (`conflictDate`, `totalHoursOnDate`, `attemptedId`) so the CLI can display precise, actionable error messages. `DeadlineConflictException` extends `Exception` (checked), forcing callers to handle it explicitly and making scheduling conflicts impossible to silently ignore.

3.5. Collections

`SchedulerManager` maintains the master task list as an `ArrayList<Task>`, which is re-sorted in-place after every mutation. An `ArrayList` was chosen over a `PriorityQueue` because it supports random-access by index (required for update and delete operations) while still allowing $O(n \log n)$ re-sorting via `Collections.sort`. Strategy switches trigger an immediate re-sort to keep the view consistent at all times.

4. Design Patterns

4.1. Strategy Pattern

Pattern: Strategy (Behavioural)

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. `SchedulerManager` selects and switches strategies at runtime without modifying any existing class — a direct application of the Open/Closed Principle.

The `SchedulingStrategy` interface exposes two methods: `sort(List<Task>)` and `getStrategyName()`. Three concrete implementations are provided:

1. **`DeadlineStrategy` — Earliest Deadline First.** Tasks due soonest appear at the top. Ties are broken by higher priority value. Best suited when the primary concern is "what is about to expire?"

2. **`PriorityStrategy` — Highest Priority First.** Tasks with priority 5 (critical) appear first; ties resolved by closer deadline. Best suited when the user must focus on importance over immediacy.

3. **`UrgencyStrategy` — Combined Urgency Score.** Uses each task's polymorphic `getUrgencyScore()` to produce a holistic rank that considers priority, deadline proximity, and type-specific factors simultaneously. The most intelligent strategy for general daily use.

4.2. Observer Pattern

Pattern: Observer (Behavioural)

Defines a one-to-many dependency. When `SchedulerManager` mutates the task list, all registered observers are notified automatically without tight coupling.

The `TaskObserver` interface declares a single callback: `onTaskListUpdated(List<Task>, String)`. Two observers are registered at startup:

1. **`DeadlineConflictNotifier`** — Scans for three conditions: overdue tasks (deadline passed), urgent deadlines (within 3 days), and overloaded days (total estimated hours > 8 h). Prints structured alerts with emoji severity indicators to the console.

2. **`WorkloadMonitor`** — Computes total pending estimated hours and classifies workload health as `LIGHT` ($\leq$10 h), `MODERATE` ($\leq$25 h), `HEAVY` ($\leq$50 h), or `OVERLOADED` (> 50 h). Reports status after every task list change.

Every mutating operation in `SchedulerManager` — add, update, delete, complete — ends with a `notifyObservers(eventName)` call. An unmodifiable snapshot is passed to prevent observers from accidentally mutating the live list.

5. Code Snippets with Explanations

5.1. Snippet 1: Abstract Task — Constructor Validation

```java
public Task(String title, String description, int priority,
            LocalDate deadline, int estimatedHours) {

    if (title == null || title.trim().isEmpty())
        throw new IllegalArgumentException("Task title cannot be empty.");
    if (priority < 1 || priority > 5)
        throw new IllegalArgumentException("Priority must be between 1 and 5.");
    if (deadline == null || deadline.isBefore(LocalDate.now()))
        throw new IllegalArgumentException("Deadline must be a future date.");
    if (estimatedHours <= 0)
        throw new IllegalArgumentException("Estimated hours must be positive.");

    this.id             = idCounter++;
```

```java
    this.title          = title.trim();
    this.priority       = priority;
    this.deadline       = deadline;
    this.estimatedHours = estimatedHours;
    this.completed      = false;
}
```

Listing 1: Task.java — Constructor with guard clauses

Explanation: Each guard clause checks one invariant independently and throws `IllegalArgumentExce` with a specific message. This *fail-fast* design means an invalid `Task` object cannot be constructed — the error surfaces immediately at the call site rather than silently propagating through the system.

5.2. Snippet 2: Polymorphic Urgency Score

```java
@Override
public double getUrgencyScore() {
    long days = daysUntilDeadline();
    // Base formula: same as OneTimeTask
    double baseScore = getPriority() * (1.0 + (1.0 / (days +
        1)) * 10);
    // Multiplier amplifies score above any normal task
    return baseScore * urgencyMultiplier;
}

@Override
public String getTaskType() {
    return "URGENT";
}
```

Listing 2: UrgentTask.java — Overridden urgency formula

Explanation: The `@Override` annotation confirms this is a runtime polymorphic method. When `UrgencyStrategy` calls `task.getUrgencyScore()` on any object, Java resolves the correct override dynamically. An `UrgentTask` with multiplier 2.0 will always outrank a same-priority `OneTimeTask`, ensuring escalated tasks surface immediately.

5.3. Snippet 3: Strategy Pattern in SchedulerManager

```java
private SchedulingStrategy strategy;

```

```java
public void setStrategy(SchedulingStrategy strategy) {
    this.strategy = strategy;
    resort();   // re-sort immediately with new algorithm
    System.out.println("Strategy switched to: "
                        + strategy.getStrategyName());
}

private void resort() {
    strategy.sort(tasks);   // polymorphic delegation
}

public void addTask(Task task) throws DeadlineConflictException
    {
    checkWorkloadCap(task, -1);
    tasks.add(task);
    resort();
    notifyObservers("TASK_ADDED");
}
```

Listing 3: SchedulerManager.java — Strategy switching at runtime

Explanation: The `strategy` field holds any object implementing `SchedulingStrategy`. Calling `strategy.sort(tasks)` delegates entirely to the current algorithm. Adding a fourth strategy requires only a new class; no modifications to `SchedulerManager` are needed — a direct application of the Open/Closed Principle.

5.4. Snippet 4: Observer Notification

```java
// SchedulerManager.java
private void notifyObservers(String event) {
    List<Task> snapshot = Collections.unmodifiableList(tasks);
    for (TaskObserver obs : observers) {
        obs.onTaskListUpdated(snapshot, event);
    }
}

// DeadlineConflictNotifier.java
@Override
public void onTaskListUpdated(List<Task> tasks, String event) {
    alertLog.clear();
    checkOverdueTasks(tasks);
```

```
    checkUpcomingDeadlines(tasks);
    checkWorkloadConflicts(tasks);
    printAlerts(event);
}
```

Listing 4: Observer pattern — notification and conflict detection

Explanation: An unmodifiable snapshot is passed so observers cannot corrupt the live list. Each observer independently decides what to analyse. This decoupling means a new observer (e.g., an email notifier or a GUI badge) can be added by simply implementing `TaskObserver` and calling `addObserver()` — no changes to `SchedulerManager` are required.

5.5. Snippet 5: Custom Checked Exception

```java
public class DeadlineConflictException extends Exception {

    private final LocalDate conflictDate;
    private final int       totalHoursOnDate;

    public DeadlineConflictException(String message,
                                     LocalDate conflictDate,
                                     int totalHoursOnDate) {
        super(message);
        this.conflictDate     = conflictDate;
        this.totalHoursOnDate = totalHoursOnDate;
    }

    @Override
    public String toString() {
        return String.format(
            "DeadlineConflictException: %s [Date: %s | Hours:
               %dh]",
            getMessage(), conflictDate, totalHoursOnDate);
    }
}
```

Listing 5: DeadlineConflictException.java — contextual exception

Explanation: Extending `Exception` (checked) forces all callers to handle or declare this exception — scheduling conflicts cannot be silently swallowed. The `conflictDate` and

`totalHoursOnDate` fields allow the CLI to display a precise message such as "Adding this task would schedule 14h on 2026-04-01 (cap: 12h/day)" rather than a generic stack trace.

6. Data Persistence

6.1. File Format: JSON

Tasks are persisted to `data/tasks.json` using a hand-written serialiser in `TaskFileHandler.java`. No external libraries are required; the implementation uses only `java.nio.file.Files` and string manipulation.

JSON was chosen over CSV because it naturally supports optional, type-specific fields (e.g., `intervalDays` is only meaningful for `RecurringTask`) without sentinel columns or positional ambiguity.

6.2. Save and Load Cycle

- **On startup:** `Main` calls `fileHandler.loadTasks()`, which reads the JSON file and reconstructs typed task objects using a `switch` expression on the `"type"` field.
- **On exit:** `fileHandler.saveTasks(manager.getAllTasks())` serialises every task, including all subclass-specific fields.
- **Robustness:** Malformed entries are skipped with a console warning rather than crashing. A missing or empty file starts a fresh empty list silently.

6.3. JSON Record Structure

```
{
  "id": 2,
  "type": "RECURRING",
  "title": "Weekly Progress Report",
  "description": "Submit to supervisor",
  "priority": 3,
  "deadline": "2026-04-07",
  "estimatedHours": 2,
  "completed": false,
  "intervalDays": 7,
  "occurrencesLeft": 10,
  "urgencyMultiplier": 1.0,
  "escalationReason": ""
}
```

Listing 6: Sample JSON record for a RecurringTask

7. Git Commit History

The following image documents incremental development spread across multiple days, demonstrating consistent progress rather than a last-minute bulk upload.

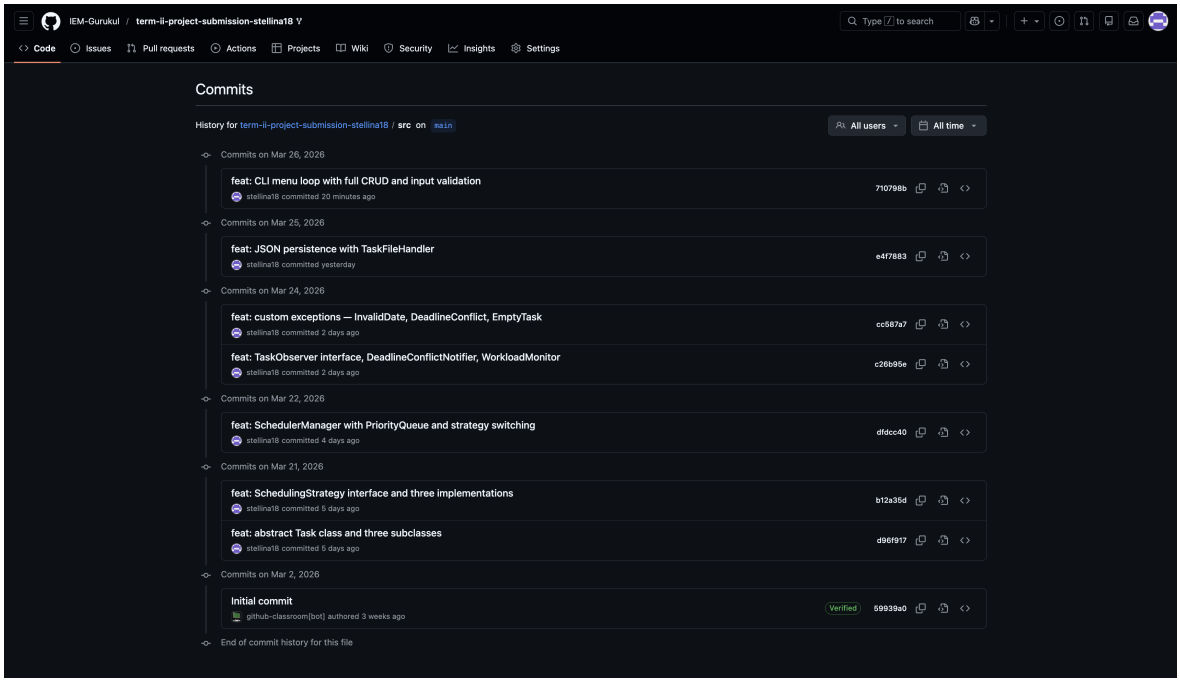

Figure 2: Git Commit History — Smart Task Scheduler

8. Challenges and Learnings

8.1. Challenges Encountered

1. Designing a clean abstraction boundary. The initial design placed urgency computation inside `SchedulerManager` as a series of `instanceof` checks. Refactoring it into an abstract method on `Task` was a key architectural insight: each task type knows its own urgency better than any external class, and this change eliminated all `instanceof` usage and made the Strategy pattern dramatically cleaner.

2. Serialisation without external libraries. Writing a JSON parser from scratch required careful handling of edge cases: escaped quotes in title strings, boolean values without surrounding quotes, and optional subclass-specific fields. The solution was a robust `extractValue()` method with per-entry `try-catch` blocks to skip corrupted records without crashing.

3. Observer notification timing. Early versions notified observers *before* re-sorting, causing `WorkloadMonitor` to display a stale task order. Moving `notifyObservers()` to after `resort()` ensured observers always receive the correctly ordered, up-to-date snapshot.

8.2. Key Learnings

- The **Strategy pattern** reduces `if-else` branching in core logic and makes adding new algorithms completely risk-free to existing code.
- **Abstract classes with partial implementation** are more powerful than interfaces alone when subclasses need to share validated, stateful behaviour.
- **Custom checked exceptions** with contextual fields produce far better user-facing error messages than re-throwing generic runtime exceptions.
- **Observer decoupling** allowed conflict detection and workload monitoring to be developed and tested entirely independently of the scheduling logic.

9. Conclusion

The Smart Task Scheduler successfully demonstrates all five core OOP principles — Abstraction, Inheritance, Polymorphism, Exception Handling, and Collections — within a cohesive, real-world application. The Strategy and Observer design patterns ensure the system is fully extensible: new scheduling algorithms or notification listeners can be added without modifying any existing class.

The layered architecture cleanly separates domain logic from persistence and presentation, mirroring professional software design conventions. The system handles edge cases gracefully — past dates, overloaded days, missing task IDs — while remaining interactive through a clean command-line interface.

The project was developed incrementally over ten days with meaningful commits at each stage, reflecting disciplined version control practices alongside sound software design.

Appendix A: APPROVED_MINOR Decision Email

Dear Megha Das,

Project: Smart Task Scheduler with Dynamic Strategy Execution
Decision: **APPROVED_MINOR**
Score: 90

Strengths: The proposal clearly identifies a relevant problem for its target audience and proposes a practical solution. It demonstrates a strong understanding of core OOP principles (Abstraction, Inheritance, Polymorphism) and articulates their application within the system's architecture. The explicit use of the Strategy Design Pattern for dynamic ordering and the Observer Pattern for conflict alerts are well-chosen and demonstrate good design foresight...

Improvements: For the Strategy Pattern, it would be beneficial to briefly outline specific scheduling strategies that will be implemented. Similarly, for the Observer Pattern, mentioning specific observers would add clarity...

Regards,
Course Instructor